Minicurso de Git e GitHub

Um guia prático para controle de versão e colaboração no desenvolvimento de software

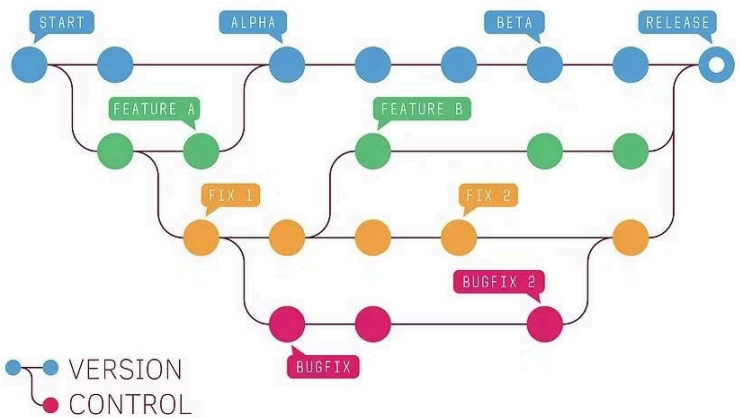
Made with GAMMA

Fundamentos

O que é controle de versão

Controle de versão é uma forma de registrar e acompanhar as mudanças feitas em arquivos ou projetos ao longo do tempo. Com ele, é possível saber quem alterou algo, quando alterou e o que foi modificado.

Isso ajuda a evitar perdas de trabalho, facilita a comparação entre versões e permite voltar para estados anteriores do projeto quando necessário.



No desenvolvimento de software, o controle de versão é essencial porque projetos mudam constantemente e, muitas vezes, são feitos por múltiplas pessoas ao mesmo tempo.

Made with GAMMA

Git vs GitHub

Made with GAMMA

Git e GitHub não são a mesma coisa

Git

A ferramenta que controla as versões do projeto no computador do usuário. Ele registra mudanças, cria históricos, permite trabalhar com branches e facilita a integração de alterações.

GitHub

Uma plataforma online que hospeda repositórios Git. Ele serve para armazenar o projeto na internet, facilitar colaboração, revisão de código, organização e compartilhamento com outras pessoas.

Em resumo: Git controla versões localmente; GitHub hospeda e conecta pessoas ao projeto.

Made with GAMMA

git init

Iniciando um novo repositório

O comando `git init` inicia um novo repositório Git dentro de um diretório. A partir dele, o Git passa a acompanhar as alterações feitas lá.

Esse comando é usado quando queremos transformar um diretório comum em um projeto versionado. Ele cria a estrutura interna necessária para o Git funcionar.

Exemplo de uso:

```
git init
```

Made with GAMMA

git add

01

Entendendo a staging area

O comando `git add` coloca alterações na área de preparação, chamada *staging area*. Isso significa que o Git passa a "preparar" os arquivos para um próximo commit.

02

Escolhendo o que preparar

Podemos adicionar um arquivo específico ou todos os arquivos alterados de uma vez. Esse passo é importante porque permite escolher exatamente o que será salvo no histórico.

03

Comandos práticos

```
git add arquivo.txt
git add .
```

O primeiro adiciona um arquivo específico, o segundo adiciona todos os arquivos alterados.

Made with GAMMA

git commit

Salvando mudanças permanentemente

O comando `git commit` salva definitivamente um conjunto de alterações no histórico do projeto. Cada commit representa um ponto de restauração, como uma fotografia do estado do projeto naquele momento.

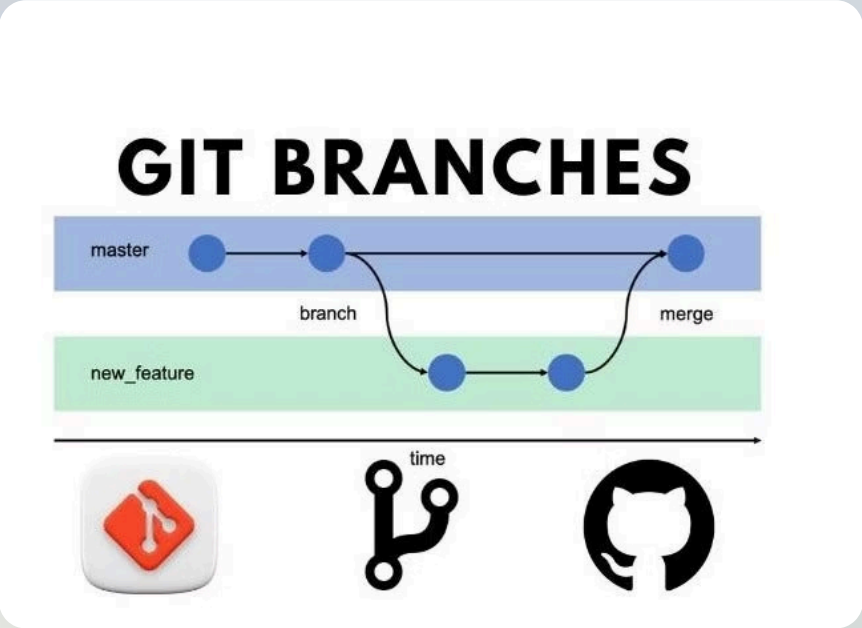
É importante escrever mensagens claras no commit, pois elas ajudam a entender o que foi feito em cada etapa.

Exemplo de uso:

```
git commit -m "Adiciona introdução ao projeto"
```

Dica: Mensagens de commit devem ser descritivas e usar o tempo presente para descrever a mudança.

Made with GAMMA



Branches

Made with GAMMA

git branch

Trabalhando com ramificações

Branches são ramificações do projeto. Elas permitem trabalhar em novas ideias, correções ou funcionalidades sem alterar diretamente a linha principal do código.

Listar branches

Mostra todos os branches existentes no repositório

```
git branch
```

Criar novo branch

Cria uma nova ramificação com o nome especificado

```
git branch
nova_funcionalidade
```

Organização segura

O comando `git branch` é muito útil para organizar o desenvolvimento de forma segura e paralela.

Made with GAMMA

git checkout

Navegando entre branches

O comando `git checkout` permite mudar de branch ou recuperar arquivos de versões anteriores. Na prática, ele é usado para navegar entre diferentes linhas de desenvolvimento.

Quando trocamos de branch, passamos a trabalhar em um contexto diferente do projeto, sem misturar alterações de outras frentes.

Exemplo de uso:

```
git checkout nova_funcionalidade
```

Pro tip: Use `git checkout -b nome_branch` para criar e mudar para um novo branch em um único comando!

Made with GAMMA

git merge

O comando `git merge` une o conteúdo de uma branch com outra. Ele é usado quando queremos trazer as mudanças feitas em uma ramificação para a branch principal ou para outra branch.

O merge é muito importante no trabalho em equipe, porque permite integrar contribuições de forma organizada.

Exemplo de uso:

```
git merge nova_funcionalidade
```

Made with GAMMA

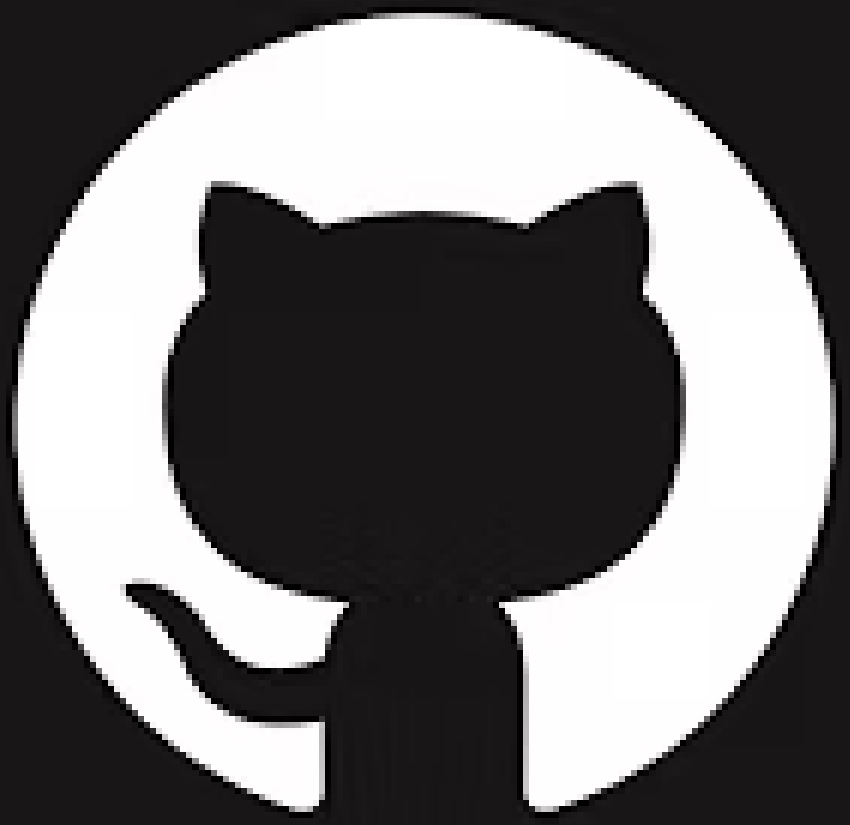
Conflitos

Conflitos acontecem quando o Git encontra alterações incompatíveis em duas branches ao tentar fazer um merge. Isso geralmente ocorre quando duas pessoas modificam a mesma parte de um arquivo ou quando mudanças diferentes afetam a mesma linha.

> ⓘ Nesses casos, o Git não decide sozinho qual versão manter. A pessoa deve analisar o conflito e escolher a solução correta manualmente.

Conflitos fazem parte do trabalho com Git e não significam erro no projeto. Eles apenas mostram que duas alterações precisam ser conciliadas.

Made with GAMMA



GitHub

GitHub é a plataforma que hospeda repositórios Git na nuvem, permitindo colaboração em tempo real entre desenvolvedores ao redor do mundo.

Made with GAMMA

Repositórios Remotos

Armazenamento na Nuvem

Um repositório remoto é a versão do projeto armazenada na internet, geralmente no GitHub.

Colaboração

Ele permite que várias pessoas acessem, baixem, enviem e acompanhem o projeto de qualquer lugar.

Backup Centralizado

O repositório remoto funciona como um ponto central de colaboração e backup.

Ele é essencial para compartilhar o trabalho com outras pessoas.

Made with GAMMA

Push e Pull

git push

Envia as alterações feitas localmente para o repositório remoto. É o comando usado para publicar atualizações no GitHub.

git pull

Faz o caminho inverso: traz para o computador as alterações que já estão no repositório remoto. Esse comando mantém o projeto local sincronizado com o que foi atualizado online.

Exemplos:

```
git push
git pull
```

Made with GAMMA

Pull Requests

Pull request é um pedido para que alterações feitas em uma branch sejam analisadas e integradas a outra branch. Ele é muito usado em colaboração, principalmente em projetos hospedados no GitHub.

01

Criar a Request

Propor mudanças de uma branch para outra

02

Revisar Código

Analise as alterações e discuta melhorias

03

Aprovar ou Rejeitar

Validar ou solicitar ajustes antes da integração

04

Integrar

Mesclar as mudanças ao projeto principal

Esse recurso permite revisar código, discutir mudanças e aprovar ou rejeitar contribuições antes da integração final. É uma prática importante para manter qualidade e organização no projeto.

Made with GAMMA

Projeto Prático

Agora vocês colocam em prática todo o conteúdo aprendido e desenvolvem um projeto real usando Git e GitHub.

Made with GAMMA

Fluxo Completo

Nesta etapa, vocês colocam em prática todo o conteúdo aprendido. O fluxo básico envolve criar um repositório, iniciar o projeto com Git, criar uma branch, fazer alterações, registrar commits, enviar para o GitHub e integrar as mudanças ao projeto principal.

Esse processo mostra como Git e GitHub funcionam juntos no dia a dia de desenvolvimento. A ideia é transformar o conteúdo teórico em experiência real.

Made with GAMMA

Colaboração entre Alunos

Criador de Branch

Um participante pode criar a branch para desenvolver uma nova funcionalidade ou correção

Revisor

Outro pode revisar o código, identificar problemas e sugerir melhorias

Testador

Um terceiro pode testar as mudanças e validar se funcionam corretamente

Acompanhamento

Todos podem acompanhar as mudanças no repositório e entender o progresso do projeto

Essa dinâmica simula um ambiente real de desenvolvimento em equipe. Vocês aprendem não só os comandos, mas também a lógica de organização, comunicação e integração de código.

Made with GAMMA